

INTERVIEW PREPARATION GUIDE

Playwright + TypeScript

Top 100 SDET Interview Questions and Answers

A practical reference covering Playwright fundamentals, TypeScript examples, test design, debugging, API testing, CI/CD, and advanced automation scenarios.

100 QUESTIONS

60 CODE EXAMPLES

INTERVIEW-READY

How to Use This Guide

Use the questions for self-testing first, then compare your response with the answer. For coding questions, explain the intent of each Playwright step—not only the syntax.

The questions are arranged in ten sets of ten for easier study. Code samples are shown in shaded blocks and can be adapted to your own Playwright test project.

Questions 1–10

Q1. What is Playwright and why is it preferred for modern TypeScript SDET automation?

ANSWER

Playwright is a browser automation and test framework that supports reliable end-to-end, API, cross-browser, and debugging workflows using TypeScript. In interviews, explain that its strongest advantages are auto-waiting, web-first assertions, isolated BrowserContext sessions, built-in test runner, tracing, parallel execution, and first-class support for Chromium, Firefox, and WebKit. A good SDET answer should also mention that Playwright encourages testing user-visible behavior instead of fragile implementation details, which improves maintainability.

Q2. Write a Playwright TypeScript test that opens a login page, fills username and password fields, submits the form, and verifies that the dashboard heading is visible.

ANSWER

```
import { test, expect } from '@playwright/test';

test('user can login successfully', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Username').fill('student@example.com');
  await page.getByLabel('Password').fill('Password@123');
  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
});
```

This test uses role and label locators because they are close to how a real user interacts with the application. The assertion is web-first, so Playwright retries until the heading appears or the timeout expires. In interviews, avoid `waitForTimeout`; prefer meaningful locators and assertions.

Q3. What is the difference between Browser, BrowserContext, and Page in Playwright?

ANSWER

Browser represents the launched browser process. BrowserContext represents an isolated session with its own cookies, local storage, permissions, viewport, and authentication state. Page represents a single tab or window inside a context. For SDET framework design, use separate contexts for independent users or roles, and separate pages for tabs or popups within the same user session.

Q4. Write a TypeScript function that removes duplicate objects from an array based on the id property.

ANSWER

```
type Entity = { id: string | number; name: string };

function removeDuplicatesById<T extends { id: string | number }>(items: T[]): T[] {
  const seen = new Set<string | number>();
  return items.filter(item => {
    if (seen.has(item.id)) return false;
    seen.add(item.id);
    return true;
  });
}

const users: Entity[] = [
  { id: 1, name: 'Asha' },
  { id: 1, name: 'Asha duplicate' },
  { id: 2, name: 'Ravi' },
];

console.log(removeDuplicatesById(users));
```

This solution is generic, preserves the first occurrence, and runs in O(n) time using a Set. In SDET interviews, explain both correctness and time complexity because automation roles often involve data preparation utilities.

Q5. Why is async/await required in Playwright TypeScript tests?

ANSWER

Most Playwright operations return Promises because browser actions, navigation, network calls, and assertions are asynchronous. `await` ensures each step completes before the next dependent step runs. Without it, tests may pass or fail unpredictably because assertions can execute before actions finish. Interview tip: mention that Playwright auto-waits for actionability, but you still need `await` to control Promise execution order.

Q6. Write a TypeScript interface for login test data and use it in a Playwright test.

ANSWER

```
import { test, expect } from '@playwright/test';

interface LoginData {
  username: string;
  password: string;
  expectedHeading: string;
}

const validUser: LoginData = {
  username: 'student@example.com',
  password: 'Password@123',
  expectedHeading: 'Dashboard',
};

test('login using typed test data', async ({ page }) => {
```

```
await page.goto('/login');
await page.getByLabel('Username').fill(validUser.username);
await page.getByLabel('Password').fill(validUser.password);
await page.getByRole('button', { name: 'Login' }).click();

await expect(page.getByRole('heading', { name: validUser.expectedHeading })).toBeVisible();
});
```

Interfaces make test data self-documenting and reduce runtime mistakes. For SDETs, this is useful when creating reusable data-driven frameworks.

Q7. What are locators in Playwright and why are they recommended over ElementHandle for most tests?

ANSWER

Locators are Playwright's high-level way to find elements at the moment of action or assertion. They support auto-waiting and retryability, so they handle dynamic UI changes better than stored element references. ElementHandle points to a specific DOM element and can become stale when the page re-renders. In interviews, say you prefer `getByRole`, `getByLabel`, `getByText`, and `getByTestId` before CSS or XPath.

Q8. Write a Playwright TypeScript test that verifies an error message appears for invalid login credentials.

ANSWER

```
import { test, expect } from '@playwright/test';

test('invalid login shows validation error', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Username').fill('wrong@example.com');
  await page.getByLabel('Password').fill('bad-password');
  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page.getByRole('alert')).toContainText('Invalid username or password');
});
```

This checks user-visible behavior instead of checking internal flags. Using `toContainText` gives some flexibility if the UI adds extra text around the error.

Q9. What are web-first assertions in Playwright?

ANSWER

Web-first assertions are Playwright assertions that automatically retry until the expected condition is met or the timeout is reached. Examples include `toBeVisible`, `toHaveText`, `toHaveURL`, `toBeEnabled`, and `toHaveCount`. They reduce flakiness because they wait for the UI state instead of forcing manual sleeps. A strong interview answer should explain that assertions are not just checks; they are synchronization points.

Q10. Write a TypeScript function that safely gets a nested property using optional chaining and nullish coalescing.**ANSWER**

```
type UserProfile = {
  user?: {
    address?: {
      city?: string | null;
    };
  };
};

function getCity(profile: UserProfile): string {
  return profile.user?.address?.city ?? 'Unknown';
}

console.log(getCity({ user: { address: { city: 'Hyderabad' } } }));
console.log(getCity({}));
```

Optional chaining prevents errors when intermediate properties are missing. Nullish coalescing returns the fallback only for `null` or `undefined`, not for valid falsy values such as an empty string.

Questions 11–20

Q11. How should a Playwright TypeScript project be structured for maintainability?**ANSWER**

A maintainable project normally separates `tests`, `pages`, `fixtures`, `test-data`, `utils`, `config`, and `reports`. Page classes should hold page behavior, tests should focus on business flows, fixtures should provide setup objects, and config should control browsers, retries, timeouts, base URL, and reporting. For interviews, emphasize that structure should support parallel execution and independent tests.

Q12. Write a Playwright test that uses a locator count assertion to verify that exactly three product cards are displayed.**ANSWER**

```
import { test, expect } from '@playwright/test';

test('product list displays three cards', async ({ page }) => {
  await page.goto('/products');
  const productCards = page.getByTestId('product-card');

  await expect(productCards).toHaveCount(3);
});
```

`toHaveCount` retries until the expected count is reached. This is better than immediately reading `await locator.count()` when the list is still loading.

Q13. Create a playwright.config.ts file that runs tests against Chromium, Firefox, and WebKit with baseURL and trace on first retry.**ANSWER**

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30_000,
  retries: process.env.CI ? 2 : 0,
  use: {
    baseURL: process.env.BASE_URL ?? 'https://example.com',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
  ],
});
```

This config keeps retries mainly for CI, captures useful diagnostics only when needed, and runs the same tests across major browser engines.

Q14. Write a TypeScript helper that accepts only admin, trainer, or student as a role and returns a dashboard URL.**ANSWER**

```
type UserRole = 'admin' | 'trainer' | 'student';

function dashboardUrl(role: UserRole): string {
  const urls: Record<UserRole, string> = {
    admin: '/admin/dashboard',
    trainer: '/trainer/dashboard',
    student: '/student/dashboard',
  };
  return urls[role];
}

console.log(dashboardUrl('admin'));
```

The union type prevents unsupported roles at compile time. This pattern is useful in role-based Playwright tests where invalid role strings can create hidden framework bugs.

Q15. What is auto-waiting in Playwright?**ANSWER**

Auto-waiting means Playwright waits for elements to be actionable before performing actions like click, fill, check, or select. It checks conditions such as visibility, stability, enabled state, and whether the element can receive events. Auto-waiting reduces flaky tests, but it does not replace waiting for business conditions such as an API response or a specific success message.

Q16. Write a Playwright TypeScript test that selects an option from a dropdown and verifies the selected value.**ANSWER**

```
import { test, expect } from '@playwright/test';

test('user can select a course category', async ({ page }) => {
  await page.goto('/courses');
  await page.getByLabel('Category').selectOption({ label: 'Playwright' });

  await expect(page.getByLabel('Category')).toHaveValue('playwright');
});
```

Use `selectOption` for native select elements. For custom dropdowns, interact with the visible button/listbox options using role-based locators.

Q17. Write a TypeScript function that calculates the total price from an array of cart items.**ANSWER**

```
type CartItem = { name: string; price: number; quantity: number };

function calculateTotal(items: CartItem[]): number {
  return items.reduce((total, item) => total + item.price * item.quantity, 0);
}

const cart: CartItem[] = [
  { name: 'Course', price: 5000, quantity: 1 },
  { name: 'Mock Interview', price: 1000, quantity: 2 },
];

console.log(calculateTotal(cart));
```

This tests arrays, objects, typing, and reduce. In an automation framework, similar utilities are used for validating UI totals against calculated expected values.

Q18. What are Playwright timeouts and how should they be managed?**ANSWER**

Playwright has multiple timeout levels: test timeout, expect timeout, action timeout, navigation timeout, and fixture timeout. Best practice is to set sensible defaults in config and override only for genuinely slower scenarios. Avoid increasing global timeouts to hide application or locator problems. During interviews, explain that timeout tuning should be based on evidence from traces and CI logs.

Q19. Write a Playwright assertion that verifies a success toast disappears after saving.**ANSWER**

```
import { test, expect } from '@playwright/test';

test('success toast disappears after save', async ({ page }) => {
  await page.goto('/profile');
  await page.getByRole('button', { name: 'Save' }).click();
```

```
const toast = page.getByRole('status').filter({ hasText: 'Saved successfully' });
await expect(toast).toBeVisible();
await expect(toast).toBeHidden();
});
```

This verifies both appearance and disappearance. It avoids fixed sleeps and lets Playwright retry the visibility state transitions.

Q20. Write a TypeScript type guard that checks whether an API response object contains a token string.

ANSWER

```
type LoginResponse = { token: string; userId: number };

function hasToken(value: unknown): value is LoginResponse {
  return (
    typeof value === 'object' &&
    value !== null &&
    'token' in value &&
    typeof (value as { token?: unknown }).token === 'string'
  );
}

const response: unknown = { token: 'abc123', userId: 10 };
if (hasToken(response)) {
  console.log(response.token);
}
```

A type guard makes runtime validation and compile-time safety work together. This is important when tests consume API responses whose shape may change.

Questions 21–30

Q21. Create a custom Playwright fixture that provides a LoginPage object to tests.

ANSWER

```
import { test as base, expect, Page } from '@playwright/test';

class LoginPage {
  constructor(private readonly page: Page) {}

  async open(): Promise<void> {
    await this.page.goto('/login');
  }

  async login(username: string, password: string): Promise<void> {
    await this.page.getByLabel('Username').fill(username);
    await this.page.getByLabel('Password').fill(password);
    await this.page.getByRole('button', { name: 'Login' }).click();
  }
}

type Fixtures = { loginPage: LoginPage };
```



```
export const test = base.extend<Fixtures>({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },
});
export { expect };
```

This fixture injects a page object into tests. It improves readability and avoids creating the same page object repeatedly in every spec.

Q22. Create a simple TypeScript Page Object Model class for a products page with methods to add an item to the cart and verify the cart count.

ANSWER

```
import { expect, Locator, Page } from '@playwright/test';

export class ProductsPage {
  private readonly cartBadge: Locator;

  constructor(private readonly page: Page) {
    this.cartBadge = page.getByTestId('cart-count');
  }

  async addProduct(name: string): Promise<void> {
    await this.page
      .getByTestId('product-card')
      .filter({ hasText: name })
      .getByRole('button', { name: 'Add to cart' })
      .click();
  }

  async expectCartCount(count: number): Promise<void> {
    await expect(this.cartBadge).toHaveText(String(count));
  }
}
```

This keeps selectors centralized and exposes business-readable methods. Some teams keep assertions in page objects, while others keep them in tests; be ready to explain your framework standard.

Q23. What is the difference between Promise.all and sequential await in Playwright tests?

ANSWER

Sequential `await` runs steps one after another. `Promise.all` starts multiple promises together and is especially useful when an action triggers an event that must be awaited at the same time, such as waiting for a popup, download, or response while clicking. In interviews, warn that `Promise.all` should not be used blindly for dependent UI actions.

Q24. Write a Playwright test that waits for an API response triggered by clicking a Save button.

ANSWER

```
import { test, expect } from '@playwright/test';

test('save profile waits for API response', async ({ page }) => {
  await page.goto('/profile');
  await page.getByLabel('Display name').fill('MSR Student');

  const [response] = await Promise.all([
    page.waitForResponse(resp =>
      resp.url().includes('/api/profile') && resp.request().method() === 'PUT'
    ),
    page.getByRole('button', { name: 'Save' }).click(),
  ]);

  expect(response.status()).toBe(200);
  await expect(page.getByRole('status')).toContainText('Profile saved');
});
```

The response wait is registered before the click, avoiding a race condition. This is a common real-time interview scenario.

Q25. How do you handle authentication efficiently in Playwright?

ANSWER

A scalable approach is to log in once through UI or API during setup, save the authenticated storage state, and reuse it in tests. This reduces execution time and avoids repeating slow login flows. Use separate storage state files for different roles and never commit real credentials. For high-value login tests, still keep a few UI login tests to validate the login feature itself.

Q26. Write a Playwright global setup that logs in and saves storage state for authenticated tests.

ANSWER

```
import { chromium, FullConfig } from '@playwright/test';

async function globalSetup(config: FullConfig): Promise<void> {
  const baseURL = config.projects[0].use.baseURL as string;
  const browser = await chromium.launch();
  const page = await browser.newPage();

  await page.goto(`${baseURL}/login`);
  await page.getByLabel('Username').fill(process.env.E2E_USER ?? '');
  await page.getByLabel('Password').fill(process.env.E2E_PASSWORD ?? '');
  await page.getByRole('button', { name: 'Login' }).click();
  await page.context().storageState({ path: 'playwright/.auth/user.json' });

  await browser.close();
}

export default globalSetup;
```

Use environment variables for credentials and save role-specific storage states. In config, set `use: { storageState: 'playwright/.auth/user.json' }` for authenticated projects.

Q27. Write a test that uses two BrowserContexts to validate admin and student sessions independently.

ANSWER

```
import { test, expect } from '@playwright/test';

test('admin and student sessions are isolated', async ({ browser }) => {
  const adminContext = await browser.newContext({ storageState: 'playwright/.auth/admin.json' });
  const studentContext = await browser.newContext({ storageState: 'playwright/.auth/student.json' });

  const adminPage = await adminContext.newPage();
  const studentPage = await studentContext.newPage();

  await adminPage.goto('/dashboard');
  await studentPage.goto('/dashboard');

  await expect(adminPage.getByRole('heading', { name: 'Admin Dashboard' })).toBeVisible();
  await expect(studentPage.getByRole('heading', { name: 'Student Dashboard' })).toBeVisible();

  await adminContext.close();
  await studentContext.close();
});
```

Separate contexts prevent cookies and local storage from leaking between roles. This is the right pattern for multi-user and role-based tests.

Q28. Write a Playwright test file that uses beforeEach to navigate to the products page before every test.

ANSWER

```
import { test, expect } from '@playwright/test';

test.beforeEach(async ({ page }) => {
  await page.goto('/products');
});

test('products heading is visible', async ({ page }) => {
  await expect(page.getByRole('heading', { name: 'Products' })).toBeVisible();
});

test('cart link is visible', async ({ page }) => {
  await expect(page.getByRole('link', { name: 'Cart' })).toBeVisible();
});
```

This keeps tests independent because each test starts from the products page. Avoid using one test to prepare data for another test.

Q29. What is the difference between `test.describe.configure({ mode: 'serial' })` and normal parallel execution?**ANSWER**

Normal Playwright execution can run tests in parallel across workers when configured. Serial mode forces tests in a describe block to run one after another and can skip remaining tests after a failure. Use serial mode only when the scenario genuinely requires ordered steps, such as a long workflow that cannot be split. For most SDET frameworks, independent parallel tests are preferred.

Q30. Write a Playwright test that handles a browser alert and verifies its message.**ANSWER**

```
import { test, expect } from '@playwright/test';

test('delete action shows confirmation alert', async ({ page }) => {
  await page.goto('/settings');

  page.once('dialog', async dialog => {
    expect(dialog.message()).toContain('Are you sure');
    await dialog.accept();
  });

  await page.getByRole('button', { name: 'Delete account' }).click();
  await expect(page.getByText('Account deletion requested')).toBeVisible();
});
```

Register the dialog handler before the action that triggers it. Otherwise the test may hang because the dialog blocks the page.

Questions 31–40

Q31. How do tags and annotations help in Playwright test execution?**ANSWER**

Tags and annotations help organize and control test execution. Tags like `@smoke`, `@regression`, or `@api` allow filtering from CLI. Annotations such as `test.skip`, `test.fixme`, `test.slow`, and custom annotations document test intent and execution constraints. In enterprise projects, tags are critical for CI pipelines, release gates, and targeted regression runs.

Q32. Write Playwright tests using tags for smoke and regression execution.**ANSWER**

```
import { test, expect } from '@playwright/test';

test('homepage loads @smoke', async ({ page }) => {
  await page.goto('/');
  await expect(page.getByRole('heading', { name: 'Welcome' })).toBeVisible();
});
```

```
});  
  
test('user can update profile @regression', async ({ page }) => {  
  await page.goto('/profile');  
  await page.getByLabel('Display name').fill('Automation User');  
  await page.getByRole('button', { name: 'Save' }).click();  
  await expect(page.getByRole('status')).toContainText('Saved');  
});
```

Run tagged tests with commands such as `npm playwright test --grep @smoke`. Tags should reflect execution purpose, not random labels.

Q33. What is APIRequestContext in Playwright?

ANSWER

APIRequestContext lets tests send HTTP requests directly from Playwright. It is useful for API testing, test data setup, cleanup, login, contract checks, and validating backend state after UI actions. Context-associated request clients can share cookies with the browser context, while standalone request contexts are isolated. SDETs should use API calls to make tests faster and less dependent on UI setup flows.

Q34. Write a Playwright API test that creates a user with POST and verifies the response status and body.

ANSWER

```
import { test, expect } from '@playwright/test';  
  
test('create user using API', async ({ request }) => {  
  const response = await request.post('/api/users', {  
    data: { name: 'Automation User', role: 'student' },  
  });  
  
  expect(response.status()).toBe(201);  
  const body = await response.json() as { id: number; name: string; role: string };  
  
  expect(body.id).toBeGreaterThan(0);  
  expect(body.name).toBe('Automation User');  
  expect(body.role).toBe('student');  
});
```

API tests are faster than UI tests and are excellent for setup and validation. Still, UI tests are needed for user journey coverage.

Q35. What is route interception and when would you use it?

ANSWER

Route interception allows a test to observe, block, continue, modify, or fulfill network requests. Use it to mock unstable third-party services, simulate error responses, validate request payloads, speed up tests, or test edge cases that are hard to create in real systems. Avoid over-mocking critical end-to-end tests because you may miss integration issues.

Q36. Write a Playwright test that mocks a products API response using page.route.**ANSWER**

```
import { test, expect } from '@playwright/test';

test('products page displays mocked products', async ({ page }) => {
  await page.route('**/api/products', async route => {
    await route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify([
        { id: 1, name: 'Playwright Course' },
        { id: 2, name: 'TypeScript Course' },
      ]),
    });
  });

  await page.goto('/products');
  await expect(page.getByText('Playwright Course')).toBeVisible();
  await expect(page.getByText('TypeScript Course')).toBeVisible();
});
```

Register the route before navigation so the first request can be intercepted. This pattern is useful for deterministic UI tests.

Q37. Write a TypeScript utility module that exports a function to generate a unique email for test data.**ANSWER**

```
// utils/test-data.ts
export function uniqueEmail(prefix = 'testuser'): string {
  return `${prefix}.${Date.now()}@example.com`;
}

// tests/register.spec.ts
import { test, expect } from '@playwright/test';
import { uniqueEmail } from '../utils/test-data';

test('register user with unique email', async ({ page }) => {
  const email = uniqueEmail('student');
  await page.goto('/register');
  await page.getByLabel('Email').fill(email);
  await page.getByLabel('Password').fill('Password@123');
  await page.getByRole('button', { name: 'Register' }).click();
  await expect(page.getByText(email)).toBeVisible();
});
```

Unique data helps avoid collisions in repeated CI runs. For large systems, prefer API-created data with cleanup over purely timestamp-based data.

Q38. Write a generic TypeScript API response wrapper and a function that extracts typed data from it.**ANSWER**

```
type ApiResponse<T> = {
  status: number;
  data: T;
  errors?: string[];
};

function getData<T>(response: ApiResponse<T>): T {
  if (response.status < 200 || response.status >= 300) {
    throw new Error(`Unexpected status: ${response.status}`);
  }
  return response.data;
}

type Course = { id: number; title: string };

const response: ApiResponse<Course> = {
  status: 200,
  data: { id: 1, title: 'Playwright with TypeScript' },
};

const course = getData(response);
console.log(course.title);
```

Generics let the same helper work for many response types without losing type information.

Q39. Configure Playwright to capture screenshots only on failure, videos on first retry, and traces on first retry.**ANSWER**

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  retries: process.env.CI ? 2 : 0,
  use: {
    screenshot: 'only-on-failure',
    video: 'on-first-retry',
    trace: 'on-first-retry',
  },
});
```

This balances diagnostic value and storage cost. Capturing everything for every test can slow execution and create large artifacts.

Q40. Write a data-driven Playwright test that validates multiple login scenarios from a typed array.**ANSWER**

```
import { test, expect } from '@playwright/test';

type LoginScenario = {
  name: string;
  username: string;
```

```
password: string;
expectedMessage: string;
};

const scenarios: LoginScenario[] = [
  { name: 'empty username', username: '', password: 'Password@123', expectedMessage: 'Username is required' },
  { name: 'empty password', username: 'user@example.com', password: '', expectedMessage: 'Password is required' },
  { name: 'invalid credentials', username: 'bad@example.com', password: 'bad', expectedMessage: 'Invalid username or password' },
];

for (const scenario of scenarios) {
  test(`login validation - ${scenario.name}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Username').fill(scenario.username);
    await page.getByLabel('Password').fill(scenario.password);
    await page.getByRole('button', { name: 'Login' }).click();

    await expect(page.getByRole('alert')).toContainText(scenario.expectedMessage);
  });
}
```

The loop creates separate tests, so reporting clearly identifies each failing scenario.

Questions 41–50

Q41. What is a hybrid Playwright framework?

ANSWER

A hybrid framework combines multiple patterns such as Page Object Model, fixtures, API helpers, data-driven testing, environment configuration, reporting, and CI integration. It is not a random collection of utilities; it should define clear responsibilities, naming standards, test data strategy, logging, error handling, and maintenance rules. In senior interviews, explain trade-offs rather than claiming one framework design fits every project.

Q42. Write a TypeScript abstract base page class and extend it with a concrete LoginPage class.

ANSWER

```
import { Page, expect } from '@playwright/test';

abstract class BasePage {
  constructor(protected readonly page: Page) {}

  async open(path: string): Promise<void> {
    await this.page.goto(path);
  }

  abstract waitForPageReady(): Promise<void>;
}
```



```
class LoginPage extends BasePage {
  async waitForPageReady(): Promise<void> {
    await expect(this.page.getByRole('heading', { name: 'Login' })).toBeVisible();
  }

  async login(username: string, password: string): Promise<void> {
    await this.page.getByLabel('Username').fill(username);
    await this.page.getByLabel('Password').fill(password);
    await this.page.getByRole('button', { name: 'Login' }).click();
  }
}
```

Abstract classes are useful when all pages share behavior but each page must implement its own readiness check. Avoid overusing inheritance when composition is simpler.

Q43. Write a TypeScript class with private locators, a public action method, and a static timeout value.

ANSWER

```
import { Locator, Page } from '@playwright/test';

export class CheckoutPage {
  static readonly DEFAULT_TIMEOUT_MS = 10_000;

  private readonly placeOrderButton: Locator;
  private readonly termsCheckbox: Locator;

  constructor(private readonly page: Page) {
    this.placeOrderButton = page.getByRole('button', { name: 'Place order' });
    this.termsCheckbox = page.getByLabel('Accept terms');
  }

  public async placeOrder(): Promise<void> {
    await this.termsCheckbox.check();
    await this.placeOrderButton.click({ timeout: CheckoutPage.DEFAULT_TIMEOUT_MS });
  }
}
```

This demonstrates encapsulation and static members. Static constants are useful for shared defaults that do not depend on object state.

Q44. Write a Playwright test that uploads a PDF file and verifies the uploaded file name is shown.

ANSWER

```
import { test, expect } from '@playwright/test';
import path from 'path';

test('user can upload a PDF document', async ({ page }) => {
  await page.goto('/documents');

  const filePath = path.join(process.cwd(), 'test-files', 'sample.pdf');
  await page.getByLabel('Upload document').setInputFiles(filePath);

  await expect(page.getByText('sample.pdf')).toBeVisible();
});
```

Use project-relative paths so the test works locally and in CI. Avoid absolute paths tied to one developer machine.

Q45. Write a Playwright test using soft assertions to validate multiple profile fields.

ANSWER

```
import { test, expect } from '@playwright/test';

test('profile displays expected user details', async ({ page }) => {
  await page.goto('/profile');

  await expect.soft(page.getByTestId('user-name')).toHaveText('Automation User');
  await expect.soft(page.getByTestId('user-role')).toHaveText('Student');
  await expect.soft(page.getByTestId('user-status')).toHaveText('Active');
});
```

Soft assertions improve feedback by showing multiple mismatches in one run. Use them only when checks are independent.

Q46. How do you handle frames and iframes in Playwright?

ANSWER

Use `frameLocator` when interacting with elements inside an iframe. It keeps locator retryability and is usually cleaner than manually fetching a Frame object. Always identify the frame using a stable selector or attribute, then locate elements inside it. In interviews, mention that iframe content has a separate DOM context.

Q47. Write a Playwright test that fills a card number inside an iframe.

ANSWER

```
import { test, expect } from '@playwright/test';

test('user enters payment card inside iframe', async ({ page }) => {
  await page.goto('/checkout');

  const paymentFrame = page.frameLocator('iframe[title="Secure payment"]');
  await paymentFrame.getByLabel('Card number').fill('4111111111111111');
  await paymentFrame.getByLabel('Expiry date').fill('12/30');
  await paymentFrame.getByLabel('CVV').fill('123');

  await expect(paymentFrame.getByText('Card accepted')).toBeVisible();
});
```

`frameLocator` keeps the test readable and avoids manually switching contexts like older tools.

Q48. Write a Playwright test that clicks a Help link opening a popup and verifies the popup title.

ANSWER

```
import { test, expect } from '@playwright/test';
```

```
test('help link opens documentation popup', async ({ page }) => {
  await page.goto('/dashboard');

  const [popup] = await Promise.all([
    page.waitForEvent('popup'),
    page.getByRole('link', { name: 'Help' }).click(),
  ]);

  await popup.waitForLoadState();
  await expect(popup).toHaveTitle(/Help|Documentation/);
});
```

Waiting for the popup and clicking in `Promise.all` prevents missing the event.

Q49. How does Playwright parallel execution work and what design mistakes make parallel tests flaky?

ANSWER

Playwright runs tests in worker processes. Tests should be independent because different workers may execute files or tests at the same time. Flakiness often comes from shared accounts, shared carts, non-unique test data, order-dependent tests, global mutable variables, and cleanup collisions. The best solution is isolated data, separate users, API setup/teardown, and avoiding serial dependencies.

Q50. Write a Playwright test that creates unique test data through API before validating it in the UI.

ANSWER

```
import { test, expect } from '@playwright/test';

test('new course created through API is visible in UI', async ({ request, page }) => {
  const title = `Course ${Date.now()}`;

  const createResponse = await request.post('/api/courses', {
    data: { title, category: 'Playwright' },
  });
  expect(createResponse.status()).toBe(201);

  await page.goto('/courses');
  await page.getByPlaceholder('Search courses').fill(title);
  await expect(page.getByRole('row', { name: new RegExp(title) })).toBeVisible();
});
```

API setup makes the test faster and less brittle than creating data through multiple UI screens. Unique names help support parallel execution.

Questions 51–60

Q51. Write a GitHub Actions workflow step that runs Playwright tests in two shards.

ANSWER

```
// Example shell commands used in separate CI jobs:
// Job 1
npx playwright test --shard=1/2

// Job 2
npx playwright test --shard=2/2
```

In a real GitHub Actions matrix, each job receives a shard index. Always upload reports, traces, and test results from every shard so failures are not lost.

Q52. Write a TypeScript PageFactory that returns LoginPage and DashboardPage objects.

ANSWER

```
import { Page, expect } from '@playwright/test';

class LoginPage {
  constructor(private readonly page: Page) {}
  async open(): Promise<void> { await this.page.goto('/login'); }
}

class DashboardPage {
  constructor(private readonly page: Page) {}
  async expectLoaded(): Promise<void> {
    await expect(this.page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
  }
}

class PageFactory {
  constructor(private readonly page: Page) {}

  loginPage(): LoginPage {
    return new LoginPage(this.page);
  }

  dashboardPage(): DashboardPage {
    return new DashboardPage(this.page);
  }
}

export { PageFactory, LoginPage, DashboardPage };
```

This pattern improves object creation consistency. For large frameworks, inject the factory as a custom fixture.

Q53. Write a Playwright visual test that masks a dynamic timestamp before comparing a screenshot.

ANSWER

```
import { test, expect } from '@playwright/test';

test('dashboard layout matches baseline', async ({ page }) => {
```

```
await page.goto('/dashboard');

await expect(page).toHaveScreenshot('dashboard.png', {
  mask: [page.getByTestId('last-updated-time')],
  animations: 'disabled',
});
});
```

Masking dynamic areas reduces false failures. Visual tests should be targeted; avoid screenshotting every page without a purpose.

Q54. Write a Playwright test that verifies keyboard navigation can reach and activate the Login button.

ANSWER

```
import { test, expect } from '@playwright/test';

test('login form supports keyboard navigation', async ({ page }) => {
  await page.goto('/login');

  await page.keyboard.press('Tab');
  await expect(page.getByLabel('Username')).toBeFocused();

  await page.keyboard.press('Tab');
  await expect(page.getByLabel('Password')).toBeFocused();

  await page.keyboard.press('Tab');
  await expect(page.getByRole('button', { name: 'Login' })).toBeFocused();
});
```

Keyboard checks catch issues that mouse-only tests miss. This is especially important for forms, menus, dialogs, and checkout flows.

Q55. What is responsive testing in Playwright?

ANSWER

Responsive testing validates that an application works across different viewport sizes and device characteristics. Playwright can configure viewports or use device descriptors for mobile emulation. Responsive tests should verify key behavior, not every pixel. Combine viewport checks with visual testing only for important layouts.

Q56. Write a Playwright test that verifies the mobile menu appears on a small viewport.

ANSWER

```
import { test, expect } from '@playwright/test';

test('mobile navigation menu opens', async ({ page }) => {
  await page.setViewportSize({ width: 390, height: 844 });
  await page.goto('/');

  await page.getByRole('button', { name: 'Open menu' }).click();
  await expect(page.getByRole('navigation')).toBeVisible();
  await expect(page.getByRole('link', { name: 'Courses' })).toBeVisible();
});
```

Viewport-based tests are simple and fast. For device-specific behavior, configure projects with Playwright device descriptors.

Q57. Add a mobile Chrome emulation project to playwright.config.ts.

ANSWER

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    { name: 'desktop-chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'mobile-chrome', use: { ...devices['Pixel 5'] } },
  ],
});
```

This allows the same tests to run against desktop and mobile-like browser settings. Keep mobile-specific assertions separate when behavior intentionally differs.

Q58. What is flaky test analysis and what steps would you follow to diagnose a flaky Playwright test?

ANSWER

Start by reproducing locally with repeat runs, then inspect trace, screenshots, videos, console logs, network logs, and CI timing. Check whether the failure is caused by weak locators, missing assertions, test data conflicts, shared state, slow backend responses, animations, retries hiding failures, or environment instability. A senior answer should end with a prevention strategy: stable locators, isolated data, API setup, deterministic waits, and flake dashboards.

Q59. Write a Playwright test that attaches custom debug information to the report using testInfo.

ANSWER

```
import { test, expect } from '@playwright/test';

test('search results include requested course', async ({ page }, testInfo) => {
  const searchTerm = 'Playwright';
  await testInfo.attach('search-term', {
    body: searchTerm,
    contentType: 'text/plain',
  });

  await page.goto('/courses');
  await page.getByPlaceholder('Search courses').fill(searchTerm);

  const resultsText = await page.getByTestId('results').innerText();
  await testInfo.attach('results-text', {
    body: resultsText,
    contentType: 'text/plain',
  });
});
```

```
    await expect(page.getByText('Playwright with TypeScript')).toBeVisible();
  });
```

Custom attachments help debug failures without rerunning the test. Attach only useful information to avoid noisy reports.

Q60. How do you design environment configuration for local, QA, staging, and production-safe test runs?

ANSWER

Use environment variables and typed config helpers to select base URL, credentials, feature flags, and test data behavior. Never hardcode secrets or destructive production actions. Production-safe tests should be read-only or use special test tenants. In interviews, emphasize separation of config from test logic and validation of required variables at startup.

Questions 61–70

Q61. Write a TypeScript config helper that reads BASE_URL and throws a clear error if it is missing.

ANSWER

```
export function requiredEnv(name: string): string {
  const value = process.env[name];
  if (!value || value.trim() === '') {
    throw new Error(`Missing required environment variable: ${name}`);
  }
  return value;
}

export const appConfig = {
  baseUrl: requiredEnv('BASE_URL'),
  username: requiredEnv('E2E_USERNAME'),
  password: requiredEnv('E2E_PASSWORD'),
};
```

Failing fast with a clear error is better than failing later with a confusing navigation or login error.

Q62. What are decorators in TypeScript and how much should an SDET know for Playwright interviews?

ANSWER

Decorators are special declarations that can annotate or modify classes, methods, properties, or parameters when the TypeScript compiler is configured for them. In Playwright frameworks, decorators may be seen in advanced logging, metadata, dependency injection, or custom reporting patterns, but they are not required for normal test writing. Interview answer: understand the concept, but prefer simple functions and fixtures unless decorators provide clear value.

Q63. Write a simple TypeScript method decorator that logs when a page object method starts and finishes.**ANSWER**

```
function logStep(
  _target: unknown,
  propertyKey: string,
  descriptor: PropertyDescriptor
): PropertyDescriptor {
  const original = descriptor.value as (...args: unknown[]) => Promise<unknown>;

  descriptor.value = async function (...args: unknown[]) {
    console.log(`Starting step: ${propertyKey}`);
    const result = await original.apply(this, args);
    console.log(`Finished step: ${propertyKey}`);
    return result;
  };

  return descriptor;
}

class LoginActions {
  @logStep
  async submitLogin(): Promise<void> {
    // page action would go here
  }
}
```

Decorators can reduce repeated logging code, but they add abstraction. Use them only when the team understands and maintains them well.

Q64. How do you test file downloads in Playwright?**ANSWER**

Register a download wait before clicking the control that starts the download. Then inspect the suggested file name or save the file to a known path. Remember that downloaded files are tied to the browser context lifecycle, so persist them before closing the context if needed. In CI, write downloads to test output directories, not random local folders.

Q65. Write a Playwright test that downloads a report and verifies the suggested filename.**ANSWER**

```
import { test, expect } from '@playwright/test';

test('user can download monthly report', async ({ page }) => {
  await page.goto('/reports');

  const [download] = await Promise.all([
    page.waitForEvent('download'),
    page.getByRole('button', { name: 'Download report' }).click(),
  ]);
});
```



```
expect(download.suggestedFilename()).toMatch(/monthly-report.*\.csv$/);
});
```

The wait must be registered before the click to avoid missing the download event.

Q66. What is `route.fulfill` versus `route.continue` in Playwright network interception?

ANSWER

`route.fulfill` completes the request with a mocked response supplied by the test. `route.continue` allows the original request to proceed, optionally with modifications such as headers or post data. Use `fulfill` for deterministic mocks and `continue` when you need to observe or slightly alter real traffic. Avoid intercepting more requests than necessary because it can make tests harder to understand.

Q67. Write a Playwright test that adds a custom header to all API requests.

ANSWER

```
import { test, expect } from '@playwright/test';

test('API requests include test header', async ({ page }) => {
  await page.route('**/api/**', async route => {
    const headers = {
      ...route.request().headers(),
      'x-test-run': 'playwright-e2e',
    };
    await route.continue({ headers });
  });

  await page.goto('/dashboard');
  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
});
```

This is useful when the backend needs to identify automation traffic. Keep such headers environment-specific and safe.

Q68. How do you handle drag and drop in Playwright?

ANSWER

Use `locator.dragTo()` for standard drag-and-drop interactions. For custom implementations, you may need mouse movements or `dataTransfer` support depending on how the app handles drag events. Always assert the final user-visible result, such as item order or board column placement, instead of only checking that the drag action completed.

Q69. Write a Playwright test that drags a task card from To Do to In Progress.

ANSWER

```
import { test, expect } from '@playwright/test';

test('task can be moved to in progress', async ({ page }) => {
```

```
await page.goto('/board');

const task = page.getByText('Prepare test plan');
const inProgressColumn = page.getByTestId('column-in-progress');

await task.dragTo(inProgressColumn);

await expect(inProgressColumn.getByText('Prepare test plan')).toBeVisible();
});
```

The final assertion validates the business outcome, not just the drag operation.

Q70. What is cross-browser testing and how should failures be triaged?

ANSWER

Cross-browser testing runs the same scenarios across Chromium, Firefox, and WebKit to catch engine-specific behavior. When a failure happens in only one browser, inspect traces, console logs, viewport differences, CSS/layout issues, and unsupported browser features. Keep a focused smoke suite for all browsers and deeper regression on the main supported browser if execution time is limited.

Questions 71–80

Q71. Write a Playwright project configuration that runs only smoke tests on all browsers.

ANSWER

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  grep: /@smoke/,
  projects: [
    { name: 'chromium-smoke', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox-smoke', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit-smoke', use: { ...devices['Desktop Safari'] } },
  ],
});
```

This keeps cross-browser feedback fast. Full regression can run on a schedule or before major releases.

Q72. How would you integrate Playwright with CI/CD?

ANSWER

Install dependencies, install browsers, run tests with CI-friendly retries and workers, publish HTML or Allure reports, upload traces/screenshots/videos, and fail the build on critical failures. Keep secrets in CI secret storage, use separate environments, and split smoke versus regression pipelines. A strong answer should mention artifact retention and trend analysis for flaky tests.

Q73. Write a GitHub Actions workflow for running Playwright TypeScript tests and uploading the HTML report.

ANSWER

```
// .github/workflows/playwright.yml
name: Playwright Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test
      - env:
          BASE_URL: ${ secrets.BASE_URL }
          E2E_USERNAME: ${ secrets.E2E_USERNAME }
          E2E_PASSWORD: ${ secrets.E2E_PASSWORD }
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
```

The report upload runs even when tests fail. This is essential for debugging CI failures.

Q74. What is the role of Docker in Playwright automation?

ANSWER

Docker provides a consistent runtime for dependencies, browsers, fonts, and OS libraries. It reduces local-versus-CI differences and simplifies execution in pipelines. Use official or well-maintained Playwright images where possible, mount reports as artifacts, and keep secrets outside the image. Docker is especially useful for large teams and reproducible CI runs.

Q75. Write a Dockerfile suitable for running Playwright TypeScript tests.

ANSWER

```
// Dockerfile
FROM mcr.microsoft.com/playwright:v1.45.0-jammy

WORKDIR /app
COPY package*.json ./
RUN npm ci

COPY . .
CMD ["npx", "playwright", "test"]
```

Pin the image version to match your Playwright package version. In real projects, update both deliberately to avoid browser/runtime mismatches.

Q76. How do HTML Report, Allure Report, and Trace Viewer differ?

ANSWER

HTML Report is Playwright's built-in test result viewer. Allure Report is a richer reporting ecosystem often used for history, categories, severity, attachments, and stakeholder-friendly dashboards. Trace Viewer is a debugging tool for individual test execution, showing actions, DOM snapshots, network, console, and timing. Use reports for test result communication and traces for root-cause analysis.

Q77. Configure Playwright to use both the built-in HTML reporter and an Allure reporter.

ANSWER

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  reporter: [
    ['html', { open: 'never' }],
    ['allure-playwright'],
    ['list'],
  ],
});
```

This gives local readability, CI console output, and richer external reporting. Ensure the Allure dependency and report generation command are added to the project scripts.

Q78. How would you design an enterprise-grade Playwright TypeScript framework from scratch?

ANSWER

Start with requirements: application architecture, supported browsers, environments, release cadence, test types, roles, data strategy, and CI gates. Then design layers: config, fixtures, page objects, API clients, test data factories, utilities, reporting, logging, and cleanup. Add standards for locator strategy, naming, tags, retries, trace collection, code review, and flake triage. A senior answer should discuss trade-offs: speed versus coverage, mocking versus integration, POM versus screen-play-like patterns, and when not to automate.

Q79. Write a custom fixture that provides an authenticated admin page and cleans up the context after the test.

ANSWER

```
import { test as base, Page } from '@playwright/test';

type AdminFixtures = {
  adminPage: Page;
};

export const test = base.extend<AdminFixtures>({
  adminPage: async ({ browser }, use) => {
```

```
const context = await browser.newContext({
  storageState: 'playwright/.auth/admin.json',
});
const page = await context.newPage();

await use(page);

await context.close();
},
});
```

This fixture isolates admin state per test and guarantees cleanup. It is safer than reusing a global page across tests.

Q80. How do you decide what to automate at UI level versus API level?

ANSWER

Automate critical user journeys, accessibility-sensitive flows, and browser-specific behavior at UI level. Use API tests for business rules, data setup, negative scenarios, contract checks, and high-volume validations. UI tests are slower and more expensive to maintain, so do not push every validation into UI. A strong SDET strategy uses a balanced test pyramid with a small number of high-value E2E tests.

Questions 81–90

Q81. Write a Playwright test that uses an API call for setup, UI for action, and API for backend validation.

ANSWER

```
import { test, expect } from '@playwright/test';

test('student enrollment updates backend status', async ({ request, page }) => {
  const courseTitle = `Course ${Date.now()}`;
  const create = await request.post('/api/courses', { data: { title: courseTitle } });
  const course = await create.json() as { id: number; title: string };

  await page.goto(`/courses/${course.id}`);
  await page.getByRole('button', { name: 'Enroll' }).click();
  await expect(page.getByText('Enrollment successful')).toBeVisible();

  const enrollment = await request.get(`/api/courses/${course.id}/enrollment`);
  expect(enrollment.status()).toBe(200);
  const body = await enrollment.json() as { enrolled: boolean };
  expect(body.enrolled).toBe(true);
});
```

This pattern keeps setup and validation fast while still testing the real user action through the UI.

Q82. How would you optimize a slow Playwright test suite?**ANSWER**

First measure where time is spent: test duration, setup, login, network, retries, and CI resource limits. Optimize by reusing authenticated storage state, moving setup to APIs, running independent tests in parallel, sharding large suites, reducing unnecessary UI steps, mocking slow external services where appropriate, and removing fixed waits. Do not optimize by weakening assertions or hiding failures with high retries.

Q83. Write a Playwright test helper that waits for a specific network response and validates status without using fixed waits.**ANSWER**

```
import { Page, expect } from '@playwright/test';

export async function waitSuccessfulResponse(
  page: Page,
  urlPart: string,
  trigger: () => Promise<void>
): Promise<void> {
  const [response] = await Promise.all([
    page.waitForResponse(resp => resp.url().includes(urlPart)),
    trigger(),
  ]);

  expect(response.status(), `Expected successful response for ${urlPart}`).toBeLessThan(400);
}
```

This helper synchronizes on a real application event. It is more reliable than adding arbitrary sleeps.

Q84. What are common anti-patterns in Playwright automation frameworks?**ANSWER**

Common anti-patterns include hardcoded sleeps, fragile XPath selectors, shared mutable test data, tests depending on previous tests, page objects with assertions and business logic mixed together without standards, excessive inheritance, over-mocking E2E tests, committing credentials, ignoring traces, and increasing timeouts instead of fixing root causes. Senior SDETs are expected to identify and remove these patterns during framework reviews.

Q85. Refactor a fragile CSS locator into a user-facing Playwright locator.**ANSWER**

```
// Fragile approach
await page.locator('#root > div:nth-child(2) .btn.primary').click();

// Better approach
await page.getByRole('button', { name: 'Submit order' }).click();

// Alternative when accessible text is not stable but test id is intentionally supported
await page.getByTestId('submit-order-button').click();
```

Role-based locators are usually more resilient and validate accessibility-friendly UI. Test IDs are acceptable when agreed as a stable automation contract.

Q86. How do you test role-based access control with Playwright?

ANSWER

Use separate authenticated storage states or API-created sessions for each role. Validate both positive and negative access: visible menus, permitted actions, blocked routes, and backend authorization responses. Keep roles isolated in separate contexts. Do not rely only on hidden UI elements; verify direct URL access and API behavior where applicable.

Q87. Write a Playwright test that verifies a student cannot access the admin page.

ANSWER

```
import { test, expect } from '@playwright/test';

test.use({ storageState: 'playwright/.auth/student.json' });

test('student cannot access admin dashboard', async ({ page }) => {
  await page.goto('/admin');

  await expect(page).toHaveURL(/access-denied|forbidden/);
  await expect(page.getByText(/not authorized|access denied/i)).toBeVisible();
});
```

For stronger coverage, add an API authorization check as well. UI-only checks can miss backend security gaps.

Q88. How would you handle test data cleanup in a parallel Playwright suite?

ANSWER

Use unique data per test and clean it up through API calls in `afterEach` or test fixtures. Prefer id-based cleanup over name-based cleanup. Cleanup should be resilient and not hide the original test failure. In some systems, using disposable tenants or scheduled cleanup jobs is safer than deleting data during tests.

Q89. Write a Playwright test that creates data through API and deletes it in `afterEach` using test-scoped variables.

ANSWER

```
import { test, expect, APIRequestContext } from '@playwright/test';

let createdCourseId: number | undefined;

test.afterEach(async ({ request }) => {
  if (createdCourseId !== undefined) {
    await request.delete(`/api/courses/${createdCourseId}`);
    createdCourseId = undefined;
  }
});
```

```
});  
  
test('course appears in UI after creation', async ({ request, page }) => {  
  const response = await request.post('/api/courses', {  
    data: { title: `Course ${Date.now()}` },  
  });  
  const body = await response.json() as { id: number; title: string };  
  createdCourseId = body.id;  
  
  await page.goto('/courses');  
  await expect(page.getByText(body.title)).toBeVisible();  
});
```

For fully parallel-safe design, keep created IDs inside fixtures or testInfo attachments rather than broad global variables when multiple tests in one file may run concurrently.

Q90. How do you review a Playwright pull request as a senior SDET?

ANSWER

Check whether tests validate meaningful user behavior, use stable locators, avoid fixed waits, isolate data, handle cleanup, run in parallel, follow naming standards, include useful assertions, and produce debug artifacts. Review page object boundaries, fixture design, config changes, and CI impact. A senior reviewer also asks whether the test belongs at UI level or should be API/unit-level.

Questions 91–100

Q91. Write a Playwright utility that retries only a non-assertion setup API call with controlled error handling.

ANSWER

```
import { APIRequestContext, expect } from '@playwright/test';  
  
export async function createCourseWithRetry(  
  request: APIRequestContext,  
  title: string,  
  attempts = 3  
): Promise<{ id: number; title: string }> {  
  let lastError: unknown;  
  
  for (let attempt = 1; attempt <= attempts; attempt++) {  
    try {  
      const response = await request.post('/api/courses', { data: { title } });  
      if (response.status() === 201) {  
        return (await response.json()) as { id: number; title: string };  
      }  
      lastError = new Error(`Status ${response.status()}`);  
    } catch (error) {  
      lastError = error;  
    }  
  }  
  
  throw new Error(`Failed to create course after ${attempts} attempts: ${String(lastError)}`);  
}
```


Controlled retries are acceptable for setup calls affected by temporary environment issues. Do not wrap product assertions with custom retries that hide real defects.

Q92. How do you use Codegen and Inspector effectively without creating poor tests?

ANSWER

Codegen and Inspector are useful for learning flows, discovering locators, and debugging actions. However, generated tests often need refactoring into page objects, role-based locators, meaningful assertions, and reusable fixtures. A strong interview answer says tools accelerate authoring but do not replace SDET design judgment.

Q93. Convert a codegen-style test into a cleaner Playwright test with meaningful locators and assertions.

ANSWER

```
import { test, expect } from '@playwright/test';

test('user searches for a course', async ({ page }) => {
  await page.goto('/courses');

  await page.getByPlaceholder('Search courses').fill('Playwright');
  await page.getByRole('button', { name: 'Search' }).click();

  await expect(page.getByRole('heading', { name: 'Courses' })).toBeVisible();
  await expect(page.getByRole('link', { name: /Playwright with TypeScript/i })).toBeVisible();
});
```

A cleaned-up test should read like a user journey. Avoid keeping recorded CSS selectors and unnecessary clicks from generated output.

Q94. How do you build confidence that Playwright tests are not giving false positives?

ANSWER

Use strong assertions tied to business outcomes, verify backend state for critical flows, avoid swallowing errors, avoid broad text matches that pass accidentally, isolate test data, and review reports for skipped or flaky tests. Also run negative tests and intentionally break small parts of the app in lower environments to confirm tests fail for the right reasons.

Q95. Write a Playwright test that validates both UI confirmation and backend order status after placing an order.

ANSWER

```
import { test, expect } from '@playwright/test';

test('order placement is reflected in backend status', async ({ page, request }) => {
  await page.goto('/cart');
```

```
await page.getByRole('button', { name: 'Place order' }).click();

const orderIdText = await page.getByTestId('order-id').innerText();
const orderId = orderIdText.replace('Order ID:', '').trim();

await expect(page.getByText('Order placed successfully')).toBeVisible();

const response = await request.get(`/api/orders/${orderId}`);
expect(response.status()).toBe(200);
const order = await response.json() as { status: string };
expect(order.status).toBe('PLACED');
});
```

Combining UI and API validation is powerful for critical flows, but use it selectively to avoid slow tests.

Q96. How would you explain Playwright architecture in a senior interview?

ANSWER

Playwright uses client APIs to control browser engines through browser-specific automation protocols. Tests run through Playwright Test, which manages workers, fixtures, projects, retries, reporters, and isolation. The key runtime objects are Browser, BrowserContext, Page, Locator, and APIRequestContext. The architecture is designed around isolated contexts, auto-waiting, and event-driven browser interaction, which makes it suitable for reliable modern web testing.

Q97. Write a minimal custom reporter that logs test titles and final status.

ANSWER

```
import type { Reporter, TestCase, TestResult } from '@playwright/test/reporter';

class SimpleStatusReporter implements Reporter {
  onTestEnd(test: TestCase, result: TestResult): void {
    console.log(`${test.title}: ${result.status}`);
  }
}

export default SimpleStatusReporter;
```

Custom reporters are useful when teams need to push results to dashboards or quality gates. Keep them lightweight and avoid slowing down the test run.

Q98. How do you approach real-time debugging when a Playwright test passes locally but fails in CI?

ANSWER

Compare local and CI differences: base URL, environment variables, browser version, OS, viewport, workers, test order, data state, network speed, permissions, and secrets. Inspect the CI trace first, then console and network logs. Try reproducing with the same command, headed/headless mode, workers, and retries. A senior SDET should avoid guessing and use artifacts to prove the root cause.

Q99. Write a Playwright test that captures console errors during execution and fails if unexpected console errors occur.

ANSWER

```
import { test, expect } from '@playwright/test';

test('dashboard has no unexpected console errors', async ({ page }) => {
  const consoleErrors: string[] = [];

  page.on('console', message => {
    if (message.type() === 'error') {
      consoleErrors.push(message.text());
    }
  });

  await page.goto('/dashboard');
  await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible();

  expect(consoleErrors).toEqual([]);
});
```

Use this carefully because some applications have known third-party console noise. Maintain an allowlist if needed.

Q100. What final advice would you give to an SDET preparing for Playwright with TypeScript interviews?

ANSWER

Prepare beyond syntax. Be ready to explain why a design is reliable, maintainable, scalable, and CI-friendly. Practice TypeScript fundamentals, locators, fixtures, POM, API testing, authentication, network mocking, parallel execution, reports, traces, and flake analysis. For senior roles, focus on trade-offs, framework architecture, test strategy, and real debugging stories from production-like pipelines.